

The Warehouse Nº 08

Arquivo Operacional para Claude + Codex

Projeto	The Warehouse Nº 08
Tipo	Sucessor espiritual original de puzzle de logística/armazém
Engine recomendada	Godot 4
Plataforma inicial	PC / Steam
Status	Documento operacional - clean-room, Claude + Codex
Data local	2026-07-09

Veredito de governança

O projeto deve avançar como jogo original. A engenharia reversa de ROM fica isolada como estudo histórico privado e não entra no repositório, nos prompts, no level design, nos assets ou no pipeline de produto.

Base proibida: Shove It! The Warehouse Game como remake, port, clone, recriação de mapas ou reimplementação por ROM

0. Veredito direto

[Certeza] O projeto The Warehouse Nº 08 deve ser desenvolvido como jogo original, não como remake técnico de Shove It! The Warehouse Game.

[Certeza] A engenharia reversa estática da ROM pode servir como estudo histórico privado, mas não deve alimentar o repositório, os mapas, os assets, o level design, o código de produção, prompts para IA ou a página da Steam.

[Certeza] Claude e Codex devem trabalhar apenas na zona limpa: regras abstratas de puzzle, GDD original, código próprio, fases autorais, arte própria, som próprio e documentação de licenças.

[Provável] A melhor estratégia técnica é Godot 4, núcleo lógico separado da apresentação visual, fases em JSON próprio, testes automatizados do core, editor interno e publicação gradual na Steam.

[Chute] O jogo terá mais chance comercial se a estética for industrial/tecnológica, com narrativa ambiental do Armazém Nº 08 e mecânicas modernas além do Sokoban puro.

1. Como usar este arquivo

Este arquivo deve ficar na raiz do repositório ou em `docs/CLAUDE\_CODEX\_WORKFLOW.md`.

Uso recomendado:

1. **Claude** lê este arquivo antes de planejar arquitetura, documentação, GDD, UX, backlog ou revisão de PR.
2. **Codex** lê este arquivo antes de implementar qualquer tarefa.
3. Nenhum agente deve trabalhar com ROM, dumps, mapas extraídos, senhas, sprites, sons ou textos do jogo original.
4. Toda tarefa precisa ter escopo pequeno, critérios de aceite e validação.
5. Toda entrega precisa informar arquivos alterados, motivo, testes executados, validação e riscos restantes.

2. Contexto do projeto

The Warehouse Nº 08 é um puzzle game original de logística industrial. O jogador controla um operador em um armazém automatizado que entrou em falha operacional. Cada sala é um setor travado do armazém e precisa ser reorganizada com lógica, planejamento espacial e eficiência de movimentos.

O projeto é inspirado em jogos clássicos de puzzle de armazém e no gênero Sokoban, mas não deve copiar Shove It! The Warehouse Game.

Frase oficial

The Warehouse Nº 08 é um puzzle game original de logística industrial, inspirado no gênero Sokoban e em clássicos de puzzle dos anos 90, com fases autorais, arte própria, música própria,

Nome de produção

- Nome visual: ****The Warehouse Nº 08****
- Nome internacional/search-friendly: ****The Warehouse No. 08****
- Slug: `the-warehouse-no-08`
- Pasta/repositório: `the-warehouse-no-08`

3. Regra jurídica central

[Certeza] "Abandonware" não é licença.

[Certeza] Jogo antigo não vira automaticamente domínio público.

[Certeza] Um remake que use nome, personagens, mapas, sprites, músicas, sons, textos, interface, capa, manual ou código do original pode ser tratado como obra derivada.

[Certeza] O caminho seguro é sucessor espiritual com identidade própria.

Permitido

- Mecânica abstrata de puzzle em grade.
- Empurrar caixas.
- Bloquear parede.
- Não puxar caixas.
- Não empurrar duas caixas.
- Alvos/zonas de entrega.
- Undo/restart.
- Eficiência por movimentos/empurrões.
- Editor de fases próprio.
- Fases originais.
- Arte original.
- Áudio original.
- Código próprio.

Proibido

- Usar ROM.
- Comitar ROM no repositório.
- Criar extractor de ROM para o jogo final.
- Usar mapas extraídos.
- Recriar as 160 salas antigas.
- Usar sprites extraídos.
- Usar músicas ou sons extraídos.

- Usar textos, senhas, nomes internos ou strings do original.
- Usar personagem Stevedore.
- Usar nome Shove It! no produto ou marketing.
- Copiar HUD, capa, manual ou identidade visual.
- Usar screenshots diretas como guia de cópia.
- Criar easter eggs com senhas ou nomes extraídos da ROM.

4. Zonas de trabalho

4.1 Zona contaminada - não entra no projeto

A zona contaminada é qualquer conteúdo extraído, copiado, convertido, derivado ou reconstituído diretamente do jogo original.

Itens proibidos:

```
ROM
.bin
.smd
offsets
dumps
mapas extraídos
senhas originais
sprites originais
sons originais
músicas originais
textos originais
screenshots diretas
layouts originais
strings internas
dados binários convertidos
```

4.2 Zona limpa - autorizada para produção

Itens permitidos:

```
GDD original
mecânicas abstratas
regras genéricas de Sokoban
level design novo
arte nova
áudio novo
UI própria
código próprio
formato JSON próprio
editor próprio
documentação de licenças
playtests
builds originais
```

4.3 Regra de isolamento

[Certeza] Claude e Codex só podem acessar a zona limpa.

[Provável] Se algum estudo de engenharia reversa existir, deve ficar fora do repositório comercial, sem virar input de IA e sem output reaproveitado.

5. Síntese técnica do estudo anterior - apenas como contexto proibido de produção

O texto analisado informa que foi feita engenharia reversa estática da ROM, sem execução em emulador. Essa informação não foi validada independentemente neste documento.

Relato técnico informado

- Jogo base estudado: Shove It! The Warehouse Game.
- Plataforma: Sega Genesis/Mega Drive.
- Núcleo: Sokoban.
- Movimento: jogador em grade.
- Caixa: empurrar uma por vez.
- Vitória: todos os alvos ocupados.
- Recursos relatados: undo, restart, trace/replay, seleção, editor, senha.

Observação crítica

[Certeza] A lógica abstrata é útil para entender o gênero.

[Certeza] Os mapas, senhas, offsets e dados da ROM não devem ser usados.

[Provável] O melhor aproveitamento técnico é reimplementar do zero um core limpo de Sokoban moderno, sem copiar layout, dados ou código.

6. Direção de game design

6.1 Premissa original

O Armazém Nº 08 era uma instalação de logística automatizada. Após uma falha no sistema central, setores inteiros foram bloqueados. O jogador assume o controle de um operador enviado para restaurar manualmente o fluxo interno. Cada sala é um problema lógico de movimentação, posicionamento e reorganização.

6.2 Tom

- Industrial.
- Tecnológico.
- Levemente misterioso.
- Minimalista.
- Inteligente.
- Tenso, mas não terror.
- Nostálgico sem copiar visual antigo.

6.3 Mecânica principal

- Grade lógica.
- Movimento em quatro direções.
- Empurrar caixas.
- Bloqueio por paredes/obstáculos.
- Alvos/zonas de entrega.
- Undo ilimitado.
- Restart rápido.
- Contador de movimentos.
- Contador de empurrões.
- Medalhas por eficiência.

6.4 Mecânicas modernas futuras

Adicionar somente depois do core sólido:

1. Caixas coloridas.
2. Portas por pressão.
3. Esteiras.
4. Piso escorregadio.
5. Caixas pesadas.
6. Caixas frágeis.
7. Robôs de rota fixa.
8. Zonas de energia.
9. Sensores.
10. Replays/fantasma da melhor tentativa.

6.5 Regra de progressão

[Certeza] Uma mecânica nova por mundo. Não misturar tudo no início.

Exemplo de mundos:

1. Recebimento - caixas normais.
2. Refrigeração - piso escorregadio.
3. Automação - esteiras.
4. Robótica - robôs de rota fixa.
5. Segurança - portas e sensores.
6. Núcleo Nº 08 - combinação avançada.

7. Arquitetura técnica recomendada

7.1 Engine

[Provável] Use Godot 4.

Motivos:

- Excelente para 2D.
- Leve.
- Boa para indie.
- Boa produtividade.
- Exporta para Windows/Linux.
- Permite separar lógica de grid da camada visual.
- Evita o peso da Unreal.

7.2 Princípio arquitetural

Separar core lógico de visual.

O core deve poder ser testado sem cena visual:

Input -> MoveCommand -> GridState -> UndoStack -> VictoryChecker -> Result

A cena visual apenas representa o estado.

7.3 Modelo de dados recomendado

Não usar o modelo antigo de soma de células como arquitetura principal. Usar camadas:

```
base_layer:
  floor
  target
  wall
  conveyor
  ice
  pressure_plate
```

```
object_layer:
  empty
  player
  crate
  heavy_crate
  fragile_crate
  robot
```

Motivo: isso escala melhor para mecânicas modernas.

8. Estrutura de repositório

```
the-warehouse-no-08/
  README.md
  .gitignore
  LICENSE
  AGENTS.md

  docs/
    00_PROJECT_VISION.md
    01_GDD.md
    02_TECHNICAL_SPEC.md
    03_LEVEL_DESIGN_GUIDE.md
    04_CLEAN_ROOM_RULES.md
    05_STEAM_RELEASE_CHECKLIST.md
    06_IP_AND_LICENSES.md
    07_PLAYTEST_PLAN.md
    08_CLAUDE_CODEX_WORKFLOW.md

  game/
```

```
project.godot

scenes/
  boot/
  menu/
  level/
  player/
  objects/
  ui/

scripts/
  core/
    Grid.gd
    Cell.gd
    Direction.gd
    MoveCommand.gd
    UndoStack.gd
    VictoryChecker.gd

  gameplay/
    PlayerController.gd
    CrateController.gd
    LevelController.gd
    LevelLoader.gd
    MedalSystem.gd

  systems/
    SaveSystem.gd
    InputSystem.gd
    AudioSystem.gd
    SettingsSystem.gd

data/
  levels/
    world_01/
  schemas/
    level.schema.json

assets/
  art/
  audio/
  fonts/
  ui/

tools/
  level_validator/
  level_editor/
  build_scripts/

tests/
  unit/
  integration/
  fixtures_clean/

marketing/
  steam/
  capsule/
  screenshots/
  trailer/
  presskit/

licenses/
  ART_LICENSES.md
  AUDIO_LICENSES.md
  FONT_LICENSES.md
  CONTRACTORS.md
```

8.1 O que não deve existir no repositório

```
roms/
shove-it.bin
*.bin
*.smd
*.gen
extracted_maps/
original_levels/
shove_it_offsets.md
passwords_original.txt
sprites_original/
music_original/
sfx_original/
```


9. .gitignore obrigatório

Adicionar ao `.gitignore`:

```
# Proibido - ROMs e dumps
roms/
*.bin
*.smd
*.gen
*.mdx
*.zip
*.7z
*.rar

# Proibido - conteúdo extraído de jogo original
extracted_maps/
original_levels/
sprites_original/
music_original/
sfx_original/
screenshots_original/
passwords_original.txt
shove_it_offsets.md

# Builds locais
builds/
exports/
*.pck
*.exe
*.app
*.dmg

# Godot
.godot/
.import/
export.cfg
export_presets.cfg

# Sistema
.DS_Store
Thumbs.db
```

Observação: se for preciso versionar builds posteriormente, usar releases/tag ou pasta controlada com política clara.

10. AGENTS.md para raiz do projeto

Copie este bloco para `AGENTS.md` na raiz:

```
# AGENTS.md - The Warehouse Nº 08

## Identidade do projeto

The Warehouse Nº 08 é um puzzle game original de logística/armazém, inspirado no gênero Sokoban e em clássicos de puzzle dos anos 90.

Este projeto NÃO é remake, port, decompilação, clone de conteúdo, recriação de mapas ou reimplementação de Shove It! The Warehouse Game.

## Regras obrigatórias

1. Não usar ROMs.
2. Não usar dumps binários.
3. Não usar sprites extraídos.
4. Não usar músicas ou sons extraídos.
5. Não usar mapas/fases extraídas.
6. Não usar senhas, textos, nomes internos ou strings do jogo original.
7. Não usar personagem Stevedore.
8. Não usar nome Shove It!.
9. Não copiar HUD, capa, manual ou identidade visual.
```

10. Não criar scripts de extração para uso no jogo final.
11. Não adicionar assets sem licença.
12. Não alterar múltiplos módulos sem justificativa.

Permitido

- Implementar mecânica abstrata de puzzle em grade.
- Implementar empurrar caixas.
- Implementar undo/restart.
- Criar fases originais.
- Criar arte original.
- Criar áudio original.
- Criar editor de fases próprio.
- Criar formato JSON próprio.

Responsabilidade do Claude

- Planejar arquitetura.
- Revisar escopo.
- Criar documentação.
- Revisar risco jurídico/técnico.
- Revisar UX.
- Quebrar tarefas para Codex.
- Revisar diffs e apontar riscos.
- Manter o projeto dentro da zona limpa.

Responsabilidade do Codex

- Implementar tarefas pequenas e testáveis.
- Criar testes.
- Refatorar sem alterar escopo.
- Não introduzir assets sem licença.
- Não criar dependência de ROM.
- Não editar múltiplos módulos sem justificativa.
- Não criar fake data enganosa.

Critério de qualidade

Toda entrega precisa informar:

- arquivos alterados;
- motivo da alteração;
- como validar;
- testes executados;
- riscos restantes;
- se foi validado por execução, teste automatizado ou apenas leitura estática.

11. Papel do Claude

Claude deve atuar como arquiteto, diretor técnico e revisor de produto.

Responsabilidades

- Manter a visão do projeto.
- Criar e revisar o GDD.
- Criar plano de produção.
- Criar issues pequenas para Codex.
- Revisar riscos jurídicos e de escopo.
- Validar UX e acessibilidade.
- Revisar arquitetura.
- Revisar PRs/diffs.
- Rejeitar qualquer conteúdo contaminado.

Claude não deve

- Pedir ROM.
- Ler mapas extraídos.
- Gerar fases baseadas em fases antigas.
- Usar senhas/textos extraídos como referência.
- Mandar Codex criar extractor.
- Aceitar commits com arquivos proibidos.

12. Papel do Codex

Codex deve atuar como implementador técnico cirúrgico.

Responsabilidades

- Implementar uma tarefa por vez.
- Criar testes.
- Refatorar com segurança.
- Manter separação entre core e visual.
- Validar comportamento por execução.
- Explicar alterações.
- Não sair do escopo.

Codex não deve

- Criar extractor de ROM.
- Usar dados do jogo original.
- Gerar assets sem licença.
- Copiar layouts antigos.
- Fazer mudança grande sem plano.
- Misturar features em um único commit.

13. Fluxo operacional Claude + Codex

Ciclo padrão

1. Claude define objetivo e risco.
2. Claude cria tarefa pequena com critério de aceite.
3. Codex implementa somente a tarefa.
4. Codex executa testes.
5. Codex entrega diff + validação.
6. Claude revisa arquitetura, IP, UX e riscos.
7. Se aprovado, merge.
8. Se houver risco, corrigir antes de avançar.

Regra de ouro

[Certeza] Claude planeja e revisa. Codex implementa e testa. Nenhum dos dois usa dados extraídos do jogo original.

14. Prompt mestre para Claude

Use este prompt no Claude:

Você é o arquiteto técnico, diretor criativo e revisor de risco do projeto The Warehouse Nº 08.

Contexto:

The Warehouse Nº 08 é um puzzle game original de logística/armazém, inspirado no gênero Sokoban e em clássicos de puzzles dos anos 90. O projeto NÃO é remake de Shove It! The Warehouse Game e não pode usar ROM, mapas, sprites, sons, músicas, textos, senhas, personagem, HUD, capa, manual ou identidade visual do jogo original.

Objetivo:

Criar documentação limpa, arquitetura técnica, GDD, plano de produção e tarefas pequenas para Codex implementar em Godot 4.

Regras obrigatórias:

- Tratar o projeto como sucessor espiritual original.
- Não pedir nem usar dados extraídos da ROM.
- Não recriar fases antigas.
- Não usar nomes ou assets do jogo original.
- Priorizar mecânica original, UX moderna, acessibilidade, suporte a controle, Steam e level design autoral.
- Classificar afirmações importantes como [Certeza], [Provável] ou [Chute].
- Quando não houver evidência, dizer que não há evidência suficiente.

Entregáveis esperados:

1. Veredito direto.
2. Riscos jurídicos, técnicos, UX, Steam e escopo.
3. Arquivos afetados.
4. Tarefas pequenas para Codex.
5. Critérios de aceite.
6. Como validar.
7. O que não foi validado.

Antes de aprovar qualquer tarefa, verifique:

- Não há ROM.
- Não há mapa extraído.
- Não há asset sem licença.
- Não há nome/personagem/texto do jogo original.
- A tarefa é pequena e testável.

15. Prompt mestre para Codex

Use este prompt no Codex:

Você é o implementador técnico do projeto The Warehouse Nº 08.

Contexto:

The Warehouse Nº 08 é um puzzle game original de logística/armazém feito em Godot 4. O projeto é inspirado no gênero Sokoban, mas NÃO é remake, port, clone ou reimplementação de Shove It! The Warehouse Game.

Proibições absolutas:

- Não usar ROM.
- Não criar extractor de ROM.
- Não usar mapas extraídos.
- Não usar sprites, músicas, sons, textos ou senhas do jogo original.
- Não usar nomes, personagens ou identidade visual do jogo original.
- Não adicionar assets sem licença.
- Não criar fake data enganosa.
- Não alterar arquivos fora do escopo sem justificar.

Objetivo inicial:

Implementar o núcleo limpo do puzzle:

- grade;
- movimento em quatro direções;
- paredes;
- caixas;
- alvos;
- empurrar uma caixa por vez;
- bloquear parede;
- bloquear duas caixas;
- undo;
- restart;

- condição de vitória;
- carregamento de fase por JSON original.

Requisitos técnicos:

- Código simples e testável.
- Separação entre lógica de grid e visual.
- Estado serializável.
- Testes para movimento, empurrão, undo e vitória.
- Fases de teste 100% originais.

Antes de codar:

1. Leia AGENTS.md.
2. Leia docs/04_CLEAN_ROOM_RULES.md.
3. Leia docs/02_TECHNICAL_SPEC.md.
4. Explique os arquivos que serão alterados.

Depois de codar:

1. Liste arquivos modificados.
2. Explique a mudança.
3. Informe comandos/testes executados.
4. Informe se foi validado por execução ou leitura estática.
5. Informe riscos restantes.

16. Backlog inicial

Sprint 0 - Blindagem jurídica e técnica

TW08-001 Criar AGENTS.md
 TW08-002 Criar docs/04_CLEAN_ROOM_RULES.md
 TW08-003 Criar docs/06_IP_AND_LICENSES.md
 TW08-004 Configurar .gitignore contra ROMs/dumps
 TW08-005 Criar README com identidade original
 TW08-006 Criar docs/08_CLAUDE_CODEX_WORKFLOW.md

Critério de aceite:

- Nenhum arquivo de ROM no repo.
- Nenhum mapa original no repo.
- Nenhuma senha original no repo.
- Nenhum nome/personagem/sprite/música de Shove It!.
- Documentação declara sucessor espiritual original.

Sprint 1 - Core lógico

TW08-010 Criar modelo de grid
 TW08-011 Criar modelo de célula
 TW08-012 Implementar movimento do jogador
 TW08-013 Implementar empurrar caixa
 TW08-014 Implementar colisão
 TW08-015 Implementar vitória
 TW08-016 Implementar undo
 TW08-017 Implementar restart
 TW08-018 Criar testes do core

Critério de aceite:

- Testes passam.
- Movimento é determinístico.
- Estado da fase serializa e desserializa.
- Undo funciona para movimento simples e empurrão.
- VictoryChecker vence quando todos os alvos estão ocupados.

Sprint 2 - Formato de fase e loader

TW08-020 Criar schema JSON de fase
 TW08-021 Criar LevelLoader
 TW08-022 Criar LevelValidator
 TW08-023 Criar 10 fases originais de MVP
 TW08-024 Criar testes de carregamento

Formato de fase recomendado:

```
{
  "id": "w01_001",
  "name": "First Shift",
  "world": 1,
  "size": { "width": 10, "height": 8 },
  "player": { "x": 2, "y": 3 },
  "tiles": [
    "#####",
    "#.....#",
    "#..T....#",
    "#..C.P...#",
    "#.....#",
    "#####",
  ],
  "par": {
    "moves": 18,
    "pushes": 5
  }
}
```

Critério de aceite:

- JSON validado por schema.
- Fase inválida não carrega.
- Fase sem alvo não passa.
- Fase com caixa sem alvo não passa.
- Fase com jogador em parede não passa.
- Fases de teste são originais.

Sprint 3 - Protótipo visual

TW08-030 Criar cena de jogo
TW08-031 Criar player placeholder
TW08-032 Criar crate placeholder
TW08-033 Criar wall/floor/target placeholders
TW08-034 Criar HUD de movimentos
TW08-035 Criar botões undo/restart
TW08-036 Criar seleção simples de fases

Critério de aceite:

- Build Windows abre.
- Controle por teclado funciona.
- Undo/restart funcionam sem bug visual.
- Vitória muda para próxima fase.
- Não há asset externo sem licença.

Sprint 4 - UX moderna e controle

TW08-040 Suporte a controle
TW08-041 Remapeamento de teclas
TW08-042 Animação suave
TW08-043 Tela de pausa
TW08-044 Configurações
TW08-045 Acessibilidade básica

Critério de aceite:

- Jogador entende a fase sem tutorial longo.
- Todos os comandos principais aparecem na UI.
- Controle e teclado funcionam.
- A fase reinicia em menos de 1 segundo.

Sprint 5 - Vertical slice

TW08-050 Direção de arte final do mundo 1
TW08-051 Música original do mundo 1
TW08-052 SFX originais

TW08-053 Sistema de medalhas
TW08-054 Save local
TW08-055 20 fases originais
TW08-056 Build Windows
TW08-057 Playtest fechado

Critério de aceite:

- 30 minutos de gameplay.
- Nenhum crash em sessão completa.
- Todas as fases são solucionáveis.
- Nenhuma fase é cópia.
- Todos os assets têm licença.

17. Critérios de bloqueio de merge

[Certeza] Qualquer item abaixo bloqueia merge:

Arquivo .bin, .smd, .gen ou ROM no repo.
Mapa extraído do jogo original.
Sprite ou áudio sem licença.
Texto ou senha original como easter egg.
Nome Shove It! no marketing.
Personagem Stevedore ou equivalente óbvio.
Fase com layout copiado.
Código que dependa de ROM.
Commit sem explicação.
Feature sem teste mínimo.
Asset sem origem documentada.
Mudança fora do escopo sem justificativa.

18. Checklist de validação técnica

Core lógico

- [] Jogador anda em quatro direções.
- [] Parede bloqueia.
- [] Caixa é empurrada se o espaço posterior estiver livre.
- [] Caixa não é puxada.
- [] Duas caixas em linha bloqueiam movimento.
- [] Caixa em alvo preserva o alvo ao sair.
- [] Vitória ocorre quando todos os alvos estão ocupados.
- [] Undo desfaz movimento simples.
- [] Undo desfaz empurrão.
- [] Restart restaura estado inicial.
- [] Estado é serializável.

Level loader

- [] JSON válido carrega.
- [] JSON inválido falha com erro claro.
- [] Fase sem jogador falha.
- [] Fase com dois jogadores falha.
- [] Fase sem alvo falha.
- [] Fase com caixa sem alvo falha.
- [] Fase com alvo sem caixa falha.
- [] Fase com player em parede falha.
- [] Fase com linha de tamanho inconsistente falha.

UX

- [] Teclado funciona.
- [] Controle funciona.
- [] Undo é visível.
- [] Restart é visível.
- [] HUD mostra movimentos.

[] HUD mostra empurrões.
[] Pausa funciona.
[] Remapeamento funciona.
[] Tela cheia/janela funciona.
[] Resolução ajustável funciona.

19. Checklist Steam

[Certeza] A Steam Direct cobra taxa por app e exige processo de revisão de loja/build. Confirmar detalhes no Steamworks antes de pagar ou agendar lançamento.

[] Conta Steamworks criada.
[] Dados fiscais e bancários preenchidos.
[] App fee pago.
[] Página Steam criada.
[] Trailer com gameplay real.
[] Screenshots reais.
[] Descrição sem prometer feature inexistente.
[] Build revisada internamente.
[] Página Coming Soon publicada com antecedência.
[] Build testada em Windows.
[] Licenças documentadas.
[] Política de privacidade se houver coleta de dados.
[] Suporte a controle validado.
[] Configurações básicas implementadas.
[] Save confiável.
[] Sem IP de terceiros não licenciado.

20. Política de assets e licenças

Todo asset precisa ter origem documentada.

Obrigatório para cada asset

Nome do arquivo:
Tipo: arte / fonte / música / SFX / UI
Autor:
Origem:
Licença:
Uso comercial permitido: sim/não
Modificação permitida: sim/não
Link/fonte:
Data de aquisição:
Observações:

Não usar

- Sprite ripado.
- Música ripada.
- SFX ripado.
- Fonte sem licença comercial.
- Asset de pacote sem licença clara.
- IA generativa sem política interna e sem documentação.

21. Modelo de issue para Codex

TW08-XXX - Título da tarefa

Objetivo

Descrever a alteração em uma frase.


```

## Escopo permitido
- Arquivo 1
- Arquivo 2

## Fora do escopo
- Não alterar X
- Não implementar Y

## Regras de clean-room
- Não usar ROM.
- Não usar dados extraídos.
- Usar apenas fases originais.

## Critérios de aceite
- [ ] Critério 1
- [ ] Critério 2
- [ ] Teste passa

## Como validar
Comandos, passos manuais ou testes automatizados.

## Relatório esperado do Codex
- Arquivos alterados
- Motivo
- Testes executados
- Resultado
- Riscos restantes

```

22. Modelo de resposta obrigatória do Codex

```

## Veredito
[Informar se concluiu ou não.]

## Arquivos alterados
- caminho/arquivo.gd - motivo

## O que foi feito
[Resumo objetivo.]

## Como validar
[Comandos e passos.]

## Testes executados
[Lista de testes.]

## Resultado
[Passou/falhou.]

## Riscos restantes
[Lista realista.]

## Validação
[Validado por execução / teste automatizado / leitura estática / não validado ainda.]

```

23. Ordem segura de execução

1. Criar documentação de blindagem.
2. Criar repositório limpo.
3. Criar `.gitignore` e `AGENTS.md`.
4. Criar GDD limpo.
5. Criar core lógico sem engine visual.
6. Criar testes do core.
7. Criar formato JSON próprio.
8. Criar 10 fases originais.
9. Criar protótipo visual.

10. Criar UX e controle.
11. Criar vertical slice.
12. Criar materiais Steam.
13. Fazer playtest.
14. Ajustar escopo.
15. Publicar Coming Soon.
16. Submeter build para revisão.

24. Primeira sequência de tarefas para executar agora

Tarefa 1 - Criar AGENTS.md

Responsável: Codex

Revisor: Claude

Criar `AGENTS.md` com regras de identidade, proibições, responsabilidades e critérios de qualidade.

Tarefa 2 - Criar CLEAN_ROOM_RULES.md

Responsável: Codex

Revisor: Claude

Criar `docs/04\_CLEAN\_ROOM\_RULES.md` separando zona contaminada e zona limpa.

Tarefa 3 - Criar README.md

Responsável: Codex

Revisor: Claude

Criar README com nome do projeto, escopo, stack, status e aviso de que é sucessor espiritual original.

Tarefa 4 - Criar .gitignore

Responsável: Codex

Revisor: Claude

Bloquear ROMs, dumps, mapas extraídos, assets originais e arquivos de build.

Tarefa 5 - Criar GDD inicial

Responsável: Claude

Implementação de arquivo: Codex

Criar `docs/01\_GDD.md` com premissa original, core loop, mecânicas, mundos, UX e escopo MVP.

25. Riscos principais

Jurídico

[Certeza] Usar material extraído do jogo original é o maior risco.

Mitigação:

- Clean-room.
- Repositório limpo.
- Assets próprios.
- Fases próprias.
- Licenças documentadas.
- Pesquisa formal de marca.
- Consulta jurídica antes da Steam.

Técnico

[Provável] O maior risco técnico é acoplar lógica de jogo à cena visual.

Mitigação:

- Core testável.
- Separação lógica/visual.
- Testes automatizados.
- Fases em JSON.

Escopo

[Provável] Editor, Workshop, rankings e mecânicas avançadas podem inflar o projeto cedo demais.

Mitigação:

- MVP pequeno.
- Vertical slice antes de produção completa.
- Uma mecânica nova por mundo.

Comercial

[Provável] Puzzle Sokoban é nichado.

Mitigação:

- Visual forte.
- Trailer claro.
- UX moderna.
- Demo jogável.
- Playtests.
- Steam page cedo.

26. O que não foi validado

- Não foi validado por advogado.
- Não foi validado no repositório real.
- Não foi validado por build Godot.
- Não foi validado por execução de testes.
- Não foi validado por análise direta da ROM.
- Não foi validado por consulta formal de marca no INPI/USPTO/WIPO.

Status correto: validado por análise documental do texto fornecido e pesquisa técnica/jurídica inicial, não por execução do projeto.

27. Fontes e referências

Fontes usadas como base documental:

1. Texto técnico fornecido pelo usuário: `Texto colado(50).txt`.
2. Steamworks - Steam Direct Fee: <https://partner.steamgames.com/doc/gettingstarted/appfee>
3. Steamworks - Release Process: <https://partner.steamgames.com/doc/store/releasing>
4. Steamworks - Steam Direct: <https://partner.steamgames.com/steamdirect>
5. U.S. Copyright Office - Circular 14, Derivative Works: <https://www.copyright.gov/circs/circ14.pdf>
6. U.S. Copyright Office - Circular 33, Works Not Protected by Copyright: <https://www.copyright.gov/circs/circ33.pdf>
7. Lei 9.610/1998 - Direitos Autorais no Brasil: https://www.planalto.gov.br/ccivil_03/leis/l9610.htm
8. Lei 9.609/1998 - Software no Brasil: https://www.planalto.gov.br/ccivil_03/leis/l9609.htm

28. Direção final

The Warehouse Nº 08 deve avançar como:

Um puzzle original de logística industrial, com mecânica de empurrar caixas, fases autorais, visual próprio, narrativa própria, áudio próprio, UX moderna, suporte a controle, editor próprio e preparação real para Steam.

A engenharia reversa fica fora da produção. O jogo comercial nasce limpo.